

The Process Abstraction

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- The process abstraction
- The process lifecycle

The process abstraction

- **Most modern OSes include the notion of a **process****
 - Frequently described as “*an instance of a program in execution*”
 - The primary means by which multiprogramming/multitasking is offered in modern operating systems

- **A process generally consists of:**
 - The program’s instructions (aka. the “program text”)
 - CPU state for the process (program counter, registers, flags, ...)
 - Memory state for the process
 - Other resources being used by the process

The process abstraction

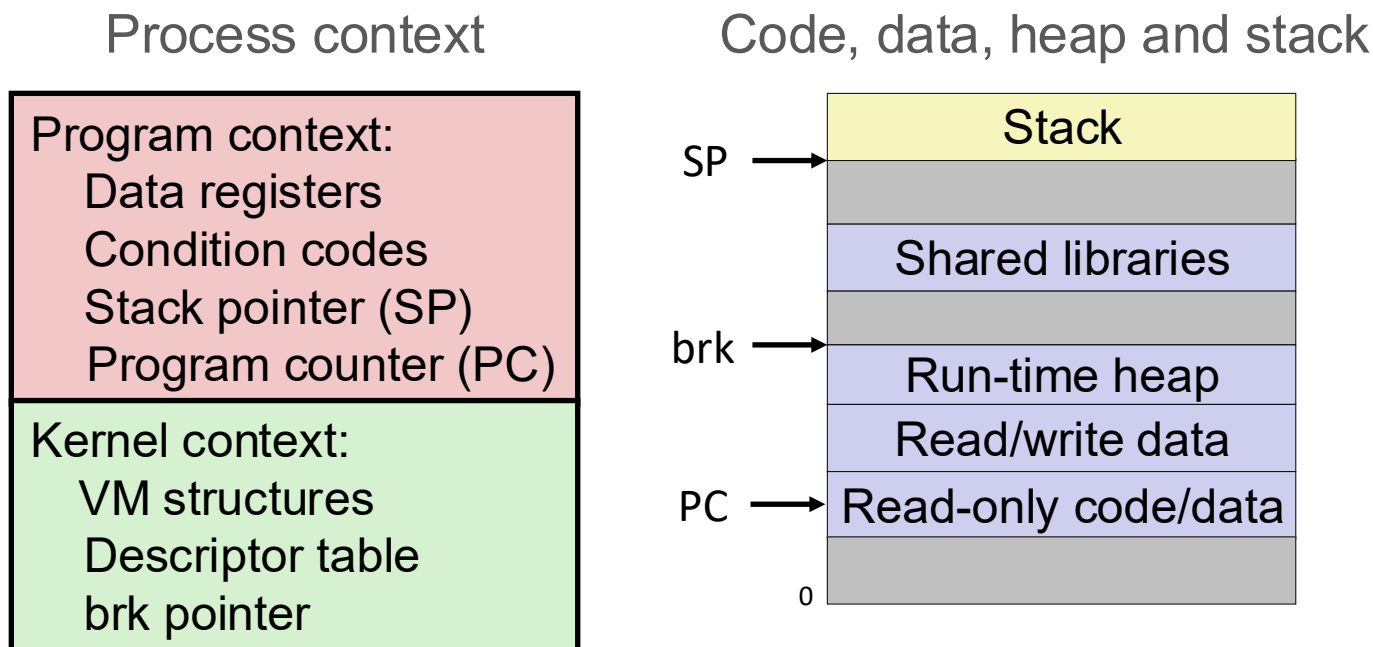
- **Which is what leads us to distinguish between programs and processes**
 - This means that one executable can have many running instances
 - Each separate instance is a different process, although each and every one of them is executing the exact same executable file
- **If we were to run this same program again, it would print a different PID**
 - Proving a new process is created whenever it is run

```
#include<stdio.h>
#include<unistd.h>

int main(){
    printf("I am the process with process-id %d\n", getpid());
}
```

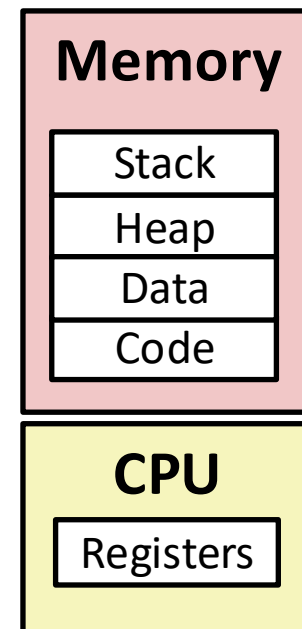
Traditional view of a process

- **Process = process context + code, data, heap and stack**



The process abstraction

- A process provides each program with two key abstractions
 - These are illusions created by the OS, not physical exclusivity
- Private address space (next lecture)
 - Each program seems to have exclusive use of main memory
 - Provided by a kernel mechanism called **virtual memory**
- Logical control flow (following lecture)
 - Each program seems to have exclusive use of the CPU, appearing to execute sequentially from its own perspective
 - Provided by a kernel mechanism called **context switching**



The process abstraction

- A primary task of the OS is to manage the entire lifecycle of processes
 - Maintain the illusion of multiple processes executing concurrently by giving each process a portion of time on the CPU
- Two big reasons why multiple processes are desirable:
 - **Performance** (responsiveness, scalability)
 - A **cleaner abstraction** for concurrent operations

Responsiveness

- Applications that perform slow or long-running tasks can perform them using **background processes**
- A **foreground process** responds to user interactions immediately
 - Responsive applications = happy users
- **Web browsers are a common example of this pattern**
 - User-interface process draws the web page, handles mouse clicks
 - A pool of background processes handles downloading of web pages from remote web servers
 - UI process updates display as downloaded files become available

Responsiveness

- Does this require multiple CPUs to yield a benefit?
 - NO!
- Background processes will usually be blocked on I/O, or waiting for work to do
 - Won't occupy the CPU
 - Similar case for UI process – waiting for user interaction
- Particularly useful in cases where most tasks are I/O bound
 - Even with a single physical processor, multiple processes can greatly improve application responsiveness

Scalability

- Instead of performing intensive computations in a single process, split it into multiple concurrently executing processes
 - E.g., a large scientific/mathematical computation
- In this scenario, processes will mostly be **CPU-bound**, not I/O-bound
 - If there is only one CPU in the system, multiple processes will probably make the program slower instead of faster
 - (extra context-switches, synchronization overhead, etc.)
- Does this require multiple CPUs to yield a benefit?
 - **YES!**
 - A single-process application cannot take advantage of multiple CPUs

A cleaner abstraction

■ Why are long-running system calls blocking, anyway?

- Why must the process wait until the request completes?

■ Example

- If a process calls `read()` and no data is available, the kernel blocks the process until data arrives

```
/* read values from pipe */  
read(fd[0], &a, sizeof(int));  
read(fd[0], &b, sizeof(int));  
  
result = a + b;
```

A cleaner abstraction

- **Blocking operations simplify programming because **control flow remains sequential****
 - Blocking operations transfer control to the called function, meaning the caller waits for the callee to finish and return control
 - The OS can schedule another process while this one waits for the operation to complete
- **Non-blocking operations, conversely, do not wait for a function to complete its task**
 - The caller can continue with other tasks without handing over control
 - Usually requires polling or event notification

Example – Iterative servers

■ Iterative servers process one request at a time

- The server accepts a client connection
- Processes the request completely
- Only then accepts the next client

```
while(1) {  
    connfd = accept(listenfd);  
    handle_request(connfd);  
    close(connfd);  
}
```

- One slow client can block the entire server, reducing responsiveness and scalability
- How can we allow the server to handle multiple clients concurrently?

Approach #1: Asynchronous I/O

■ One alternative is **asynchronous** (non-blocking) operations

- Initiate a long-running operation in the system
- Periodically check to see if the operation is complete
- If not, do other things while you wait
- When operation finally completes, go on to next steps in your task

■ Most systems provide asynchronous I/O APIs alongside blocking I/O APIs

- The call returns immediately

```
fcntl(fd, F_SETFL, O_NONBLOCK);  
n = read(fd, buf, n);
```

```
/* if no data is available read() returns -1 and errno is set to EAGAIN */
```

Approach #1: Asynchronous I/O

- **Primarily used for asynchronous networking I/O**

- `select(int nfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, timeval *timeout)`
- `poll(pollfd *fds, nfd_t nfd, int timeout)`

- **Allow a single process to sleep until one of many file descriptors becomes ready, enabling efficient event-driven servers**

- The process only runs when work exists, avoiding thousands of processes, context switching overhead, and large memory usage

Approach #1: Asynchronous I/O

■ Key idea:

- The server blocks only inside `select()`
- The kernel wakes the process only when some descriptor becomes ready
- The server then handles only the ready connections

```
while(1) {  
    select(maxfd+1, &readfds, NULL, NULL, NULL);  
  
    for(fd = 0; fd <= maxfd; fd++){  
        if(FD_ISSET(fd, &readfds)){  
            read(fd, buf, sizeof(buf));  
            handle_request(fd);  
        }  
    }  
}
```


Approach #1: Asynchronous I/O

- **Applications usually use non-blocking I/O when they want to achieve very high performance**
 - OSes heavily optimize these functions to be fast and scalable
 - Avoids a significant amount of overhead and space requirements
- **Often the fastest possible approach for high-concurrency I/O-bound workloads**
 - e.g., NGINX (“engine-x”) web server, Redis, Node.js runtime
 - Easily supports 10000+ concurrent connections (C10K problem)
 - Used by Facebook, Dropbox, Wikipedia, Wordpress, etc.

Approach #1: Asynchronous I/O

- **Non-blocking I/O in a single process is significantly harder to implement correctly**
 - The program must manage many connections simultaneously
 - Control flow is no longer sequential
 - The application must track the state of each connection
 - Operations may need to be split into multiple steps (read → process → write)
 - The program must react to events instead of executing a simple call sequence

- **The program becomes an event-driven system**
 - Often implemented as an event loop or state machine, which is harder to design, implement, and debug than a simple blocking model

Approach #2: Process-based servers

■ Key idea:

- The server accepts a connection
- It creates a child process with `fork()`
- The child handles the client request
- The parent immediately returns to `accept()` to serve other clients

```
while(1) {  
    connfd = accept(listenfd, NULL, NULL);  
  
    /* each client is handled by a separate child process */  
    if(fork() == 0) {  
        close(listenfd);  
        handle_request(connfd);  
        close(connfd);  
        exit(0);  
    }  
  
    close(connfd);  
    while(waitpid(-1, NULL, WNOHANG) > 0); /* reap any terminated children (avoid zombies) */  
}
```

Pros and Cons of process-based servers

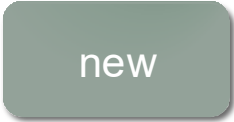
- + Handle multiple connections concurrently**
- + Clear resource isolation model**
 - File descriptors (no)
 - File tables (yes)
 - Global variables (no)
- + Simple and straightforward synchronous approach**
- Additional overhead for process control**
- Nontrivial to share data between processes**
 - (This example is too simple to demonstrate)

Overview

- The process abstraction
- **The process lifecycle**

The process lifecycle

- The process lifecycle can be represented as a state diagram
- Processes must be **created** before they can run
 - Created by the operating system at startup (e.g., in Linux by `systemd`)
 - Created when a user invokes a program via command line or GUI
 - Created when a process spawns another process



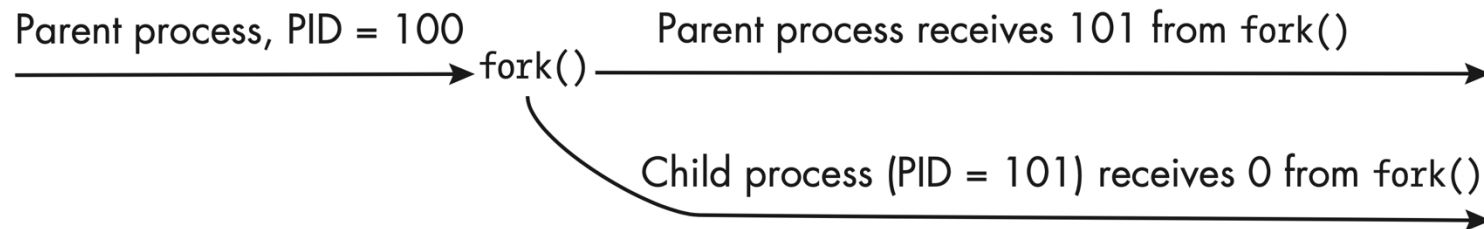
new

Process creation

- In UNIX like OSes, the `fork()` system call creates a **clone** process of the parent process
- A child process is **almost identical** to its parent
 - Gets an identical (but separate) copy of the parent's virtual address space
 - Gets identical copies of the parent's open file descriptors
 - Has a different PID than the parent
- We need to understand what's the same and what's different to avoid some common programming pitfalls
 - Explore the code samples available on Moodle

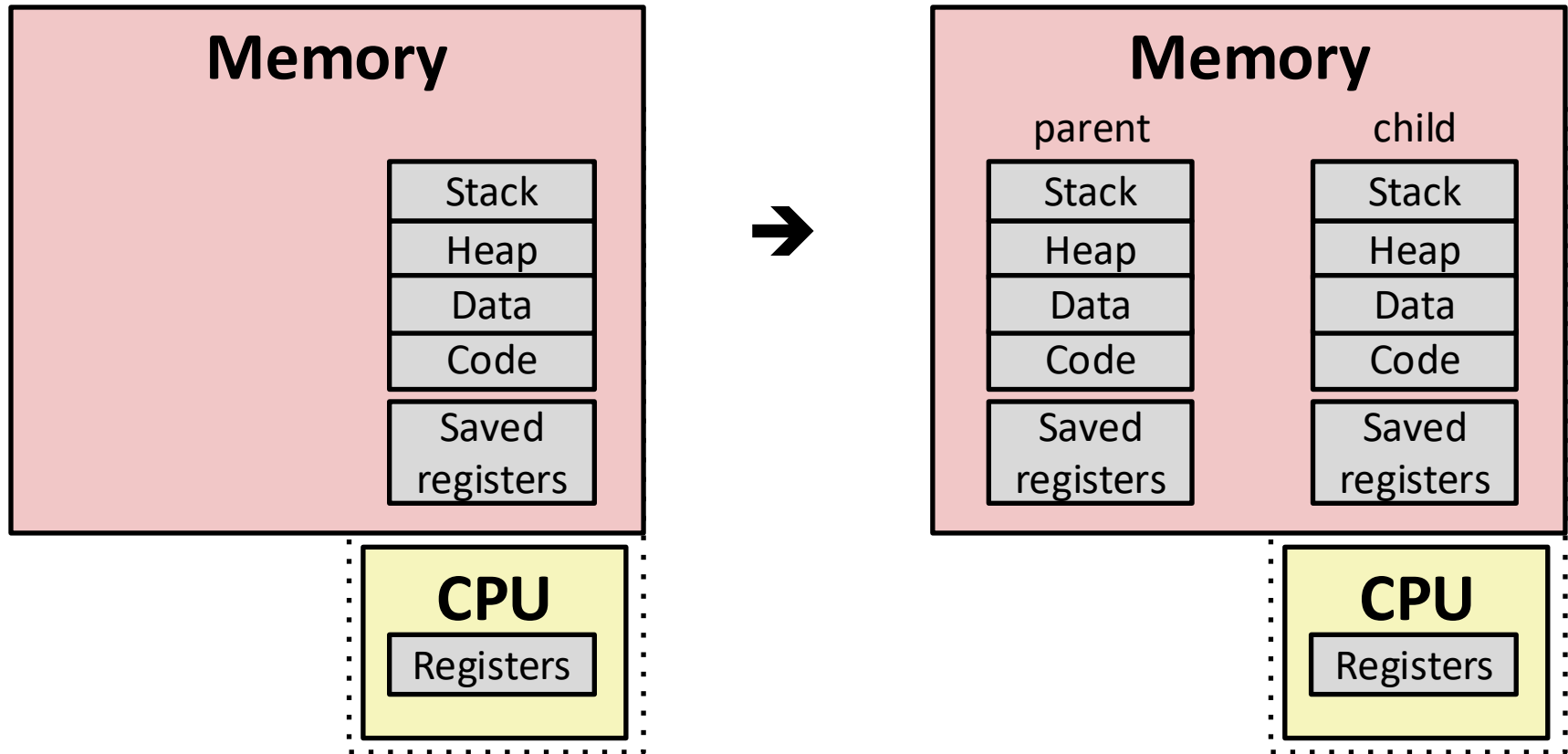
Process creation

- **`fork()` is interesting (and often confusing) because it is called once but returns twice**
 - Returns 0 to the child process, child's PID to parent process



- **When the child process is created, the kernel logically duplicates the parent's address space**
 - In practice, most systems implement this using copy-on-write, so physical memory pages are only copied if one of the processes modifies them

Conceptual view of `fork()`



Process creation

- In modern Linux systems, the **copy-on-write** mechanism optimizes memory management
- It allows parent and child processes to **share the same memory pages until one tries to modify that shared memory**
 - Rather than re-copying the entire page, the OS creates a copy that can be changed without changing the original
 - Other processes continue to refer to the original shared page

Process creation

- Among the items that are copied into the child are all of the open file descriptors from the parent
 - Recall that each process has a table of open file descriptors and that these descriptors are pointers into the kernel's table of open file descriptions

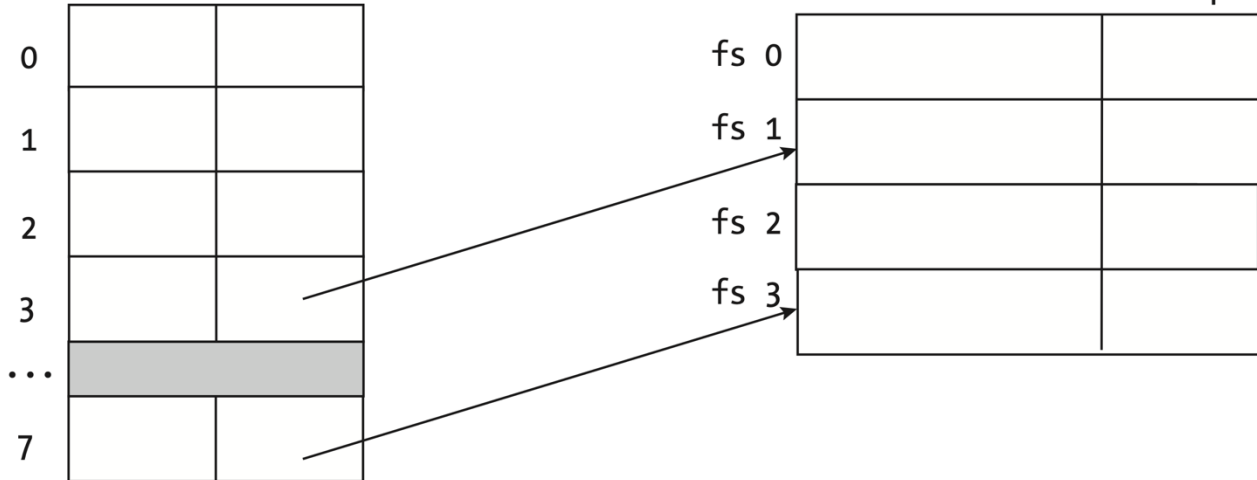
Before the
call to fork():

**Process P's file
descriptor table**

	Flags	File ref.
0		
1		
2		
3		
...		
7		

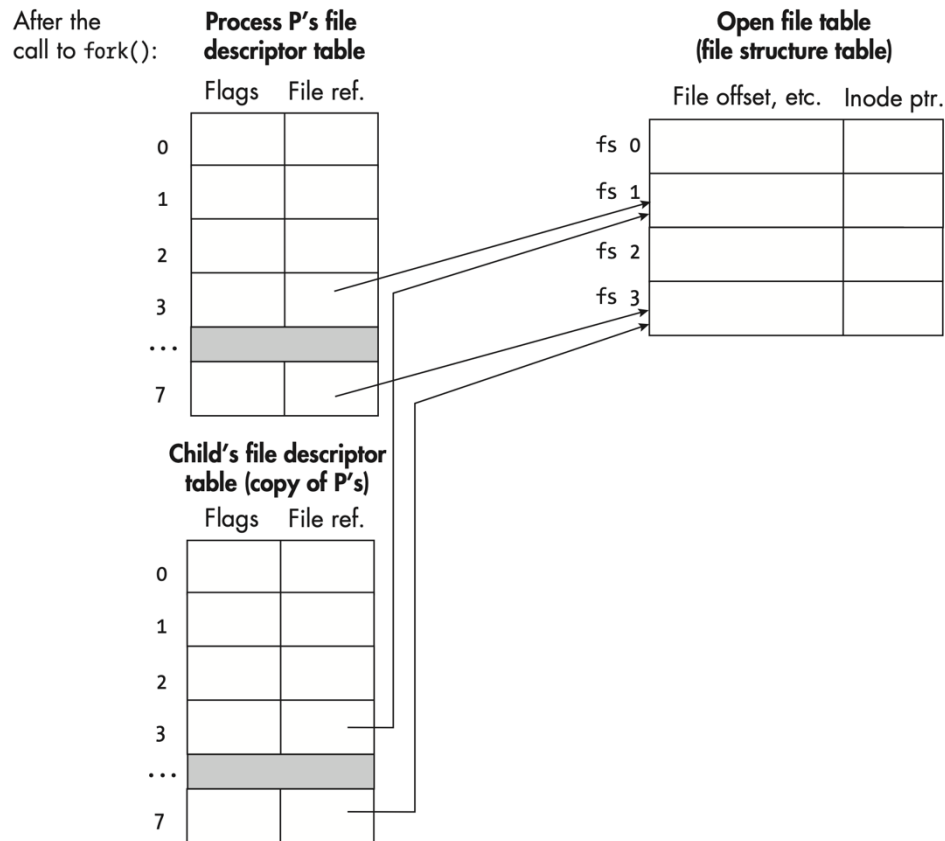
**Open file table
(file structure table)**

	File offset, etc.	Inode ptr.
fs 0		
fs 1		
fs 2		
fs 3		



Process creation

- The fact that the child process has access to the same open file descriptors as the parent is going to play an important role in inter-process communication



The process lifecycle

- Generally, there is a set of processes that can make progress
 - i.e., not waiting for I/O to complete, etc.
- Only **one process** may be **running** on each CPU at a time
 - When a process is in running state, it **holds the CPU**
 - Other processes that could run, but don't currently have the CPU, are in the **ready** state

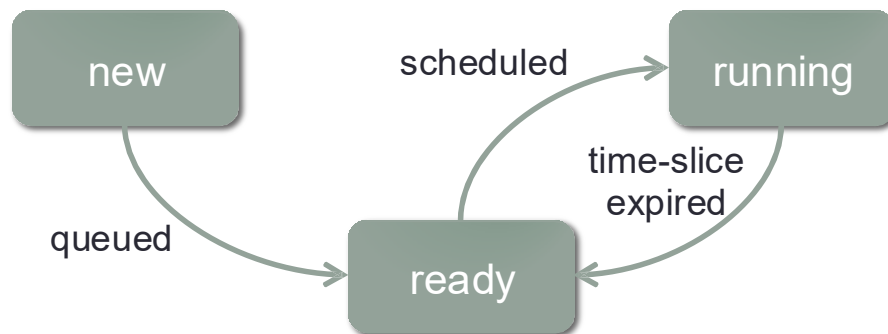
new

running

ready

The process lifecycle

- **New processes don't necessarily get the CPU right away**
 - They initially go into the collection of ready processes
- **The OS only allows the currently running process to hold the CPU for a specific amount of time**
 - When time-slice expires, running process is **preempted**, and the scheduler chooses another process from the ready queue to access the CPU



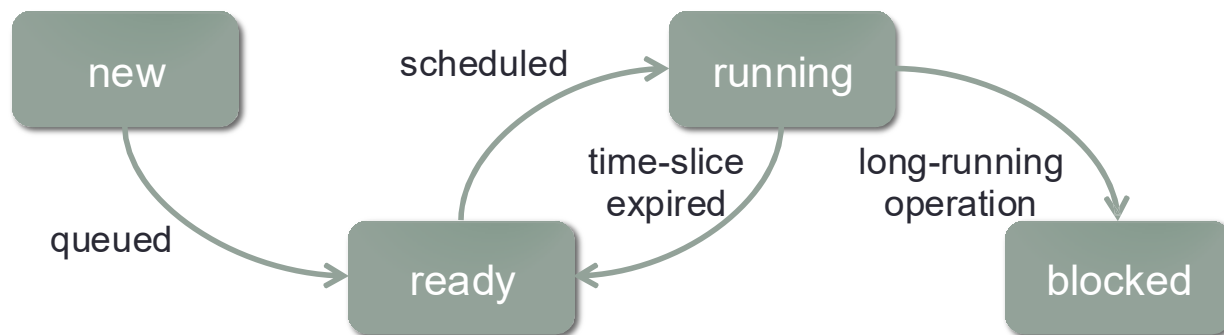
The process lifecycle

■ Processes often perform long-running tasks

- e.g., read from hard disk, network, or some other external device
- e.g., process waits for another process (e.g., a signal or termination)

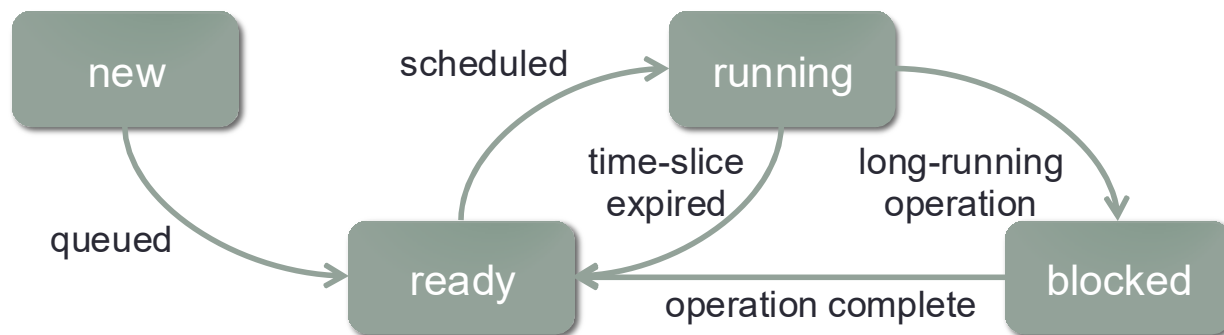
■ The process becomes **blocked** until the resource is available

- OS removes process from CPU, and chooses another ready process to run, instead of holding everyone up



The process lifecycle

- When the long-running task is completed, the blocked process can **resume** execution
 - Process is moved back into the ready state
 - Will eventually be chosen by the OS to run on the CPU again



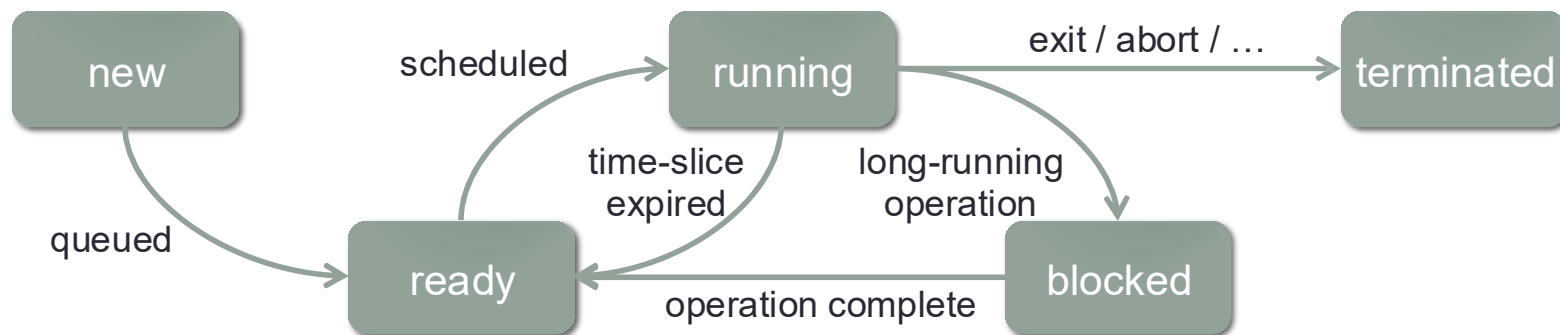
The process lifecycle

■ Processes eventually terminate

- May live for seconds, hours, months, ...

■ Several tasks must be completed at process termination

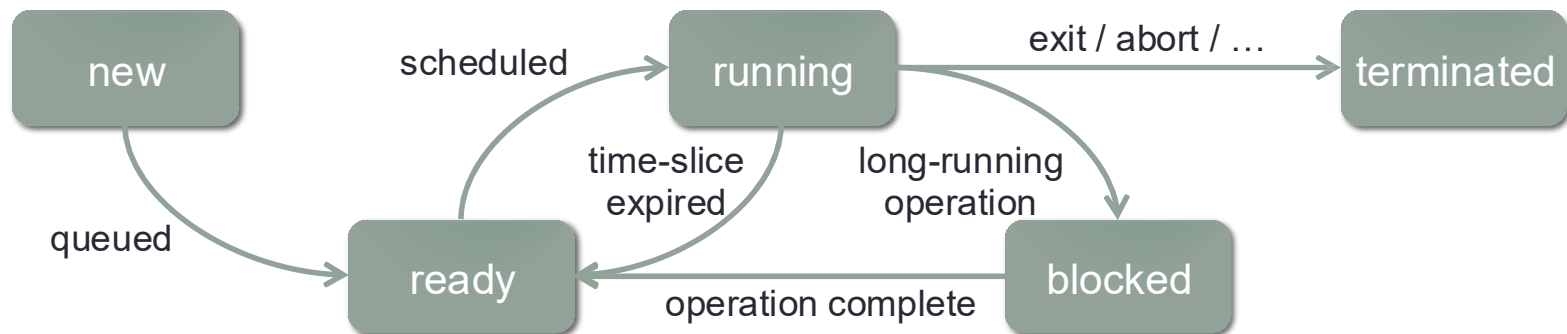
- Any “at-exit” operations must be performed
- Reclaim resources the process is still holding
- Other processes may need to observe terminating process' status



The process lifecycle

■ Voluntary termination

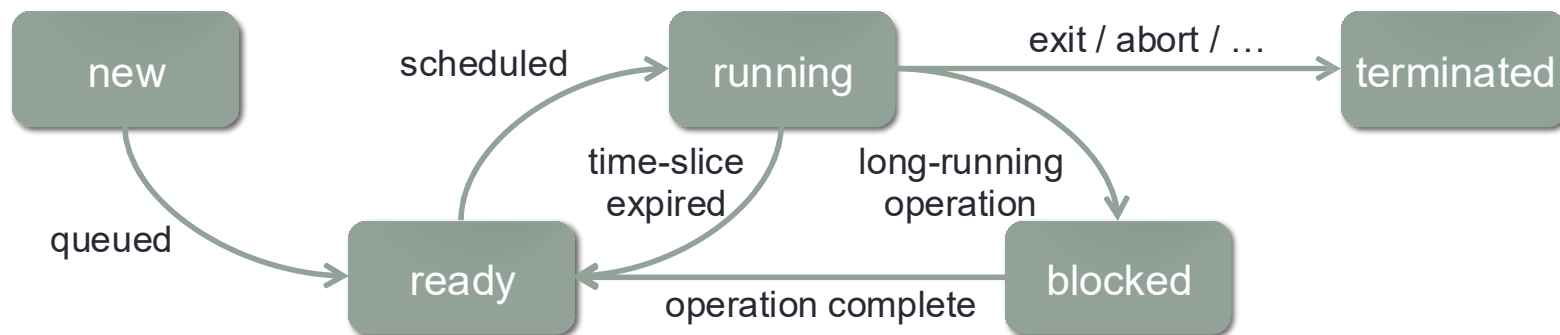
- By the process itself
- e.g., it calls `exit()` or returns from `main()`, either with success or error status



The process lifecycle

■ Involuntary termination

- Due to an unrecoverable fault in the process (e.g., segmentation fault due to dereferencing a NULL pointer)
- Due to a signal from another process (e.g., another process issues a `SIGINT`, `SIGTERM` or `SIGKILL`)



Process termination

- In UNIX like OSes, when a process has completed its execution or is terminated, the kernel sends a `SIGCHLD` signal to its parent and the process goes into the **zombie** state
- A zombie is not a running process
- The zombie process will remain in this state until the parent process clears it off from the process table
 - If the parent dies before its children, the PID 1 (the `init/systemd`) process will take ownership of them (reaps orphaned children)
 - In Linux, intermediate subreaper processes may adopt and reap orphaned children before they reach `systemd`

Zombie example

```
int main() {
    if (fork() == 0) {
        printf("Terminating Child, PID = %d\n", getpid());
        exit(0);
    } else {
        printf("Running Parent, PID = %d\n", getpid());
        while (1);    /* Busy loop, intentionally incorrect */
    }
}
```

```
linux> ./forks&
```

```
[1] 6639
```

```
Running Parent, PID = 6639
```

```
Terminating Child, PID = 6640
```

```
linux> ps
```

PID	TTY	TIME	CMD
6585	ttyp9	00:00:00	tcsh
6639	ttyp9	00:00:03	forks
6640	ttyp9	00:00:00	forks <defunct>
6641	ttyp9	00:00:00	ps

■ ps shows child process as “defunct” (i.e., a zombie)



```
linux> kill 6639
```

```
[1] Terminated
```

```
linux> ps
```

PID	TTY	TIME	CMD
6585	ttyp9	00:00:00	tcsh
6642	ttyp9	00:00:00	ps



■ Killing parent allows child to be reaped by systemd

Process termination

- **When process terminates, it still consumes system resources**
 - It preserves, at the very least, its slot in the kernel's process table, its PID, termination status, and resource usage information
- **To clear the terminated child process off the process table, the parent process must use the `wait()` / `waitpid()` system calls**
 - Suspends current process until one of its children terminates
 - Returns PID of terminated child, records exit status
 - `waitpid()` is just a more flexible version of `wait()`
- **By calling one of the wait functions, parent processes tell the kernel that it can release these resources**

Process termination

■ If multiple children completed, will take in arbitrary order

- We can use macros `WIFEXITED` and `WEXITSTATUS` to get information about exit status

```
int main() {
    pid_t pid[N];
    int i, child_status;

    for(i=0; i<N; i++){
        if ((pid[i]=fork())==0)
            exit(100+i);
    }

    for(i=0; i<N; i++){
        pid_t wpid = wait(&child_status);
        if(WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

Process termination

- **Suspends parent process until a specific process terminates**
 - See man page for several other options

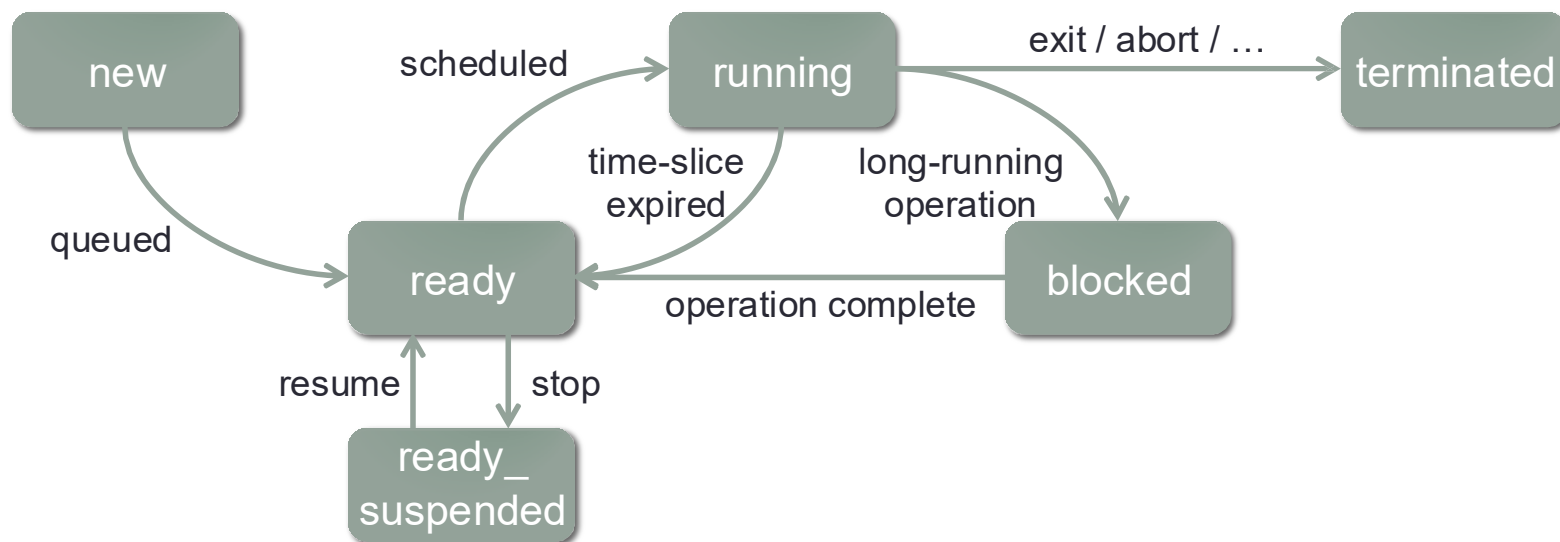
```
int main() {
    pid_t pid[N];
    int i, child_status;

    for(i=0; i<N; i++){
        if ((pid[i]=fork())==0)
            exit(100+i);
        /* Child */
    }

    for(i=0; i<N; i++){
        /* Parent */
        pid_t wpid = waitpid(pid[i], &child_status, 0);
        if(WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

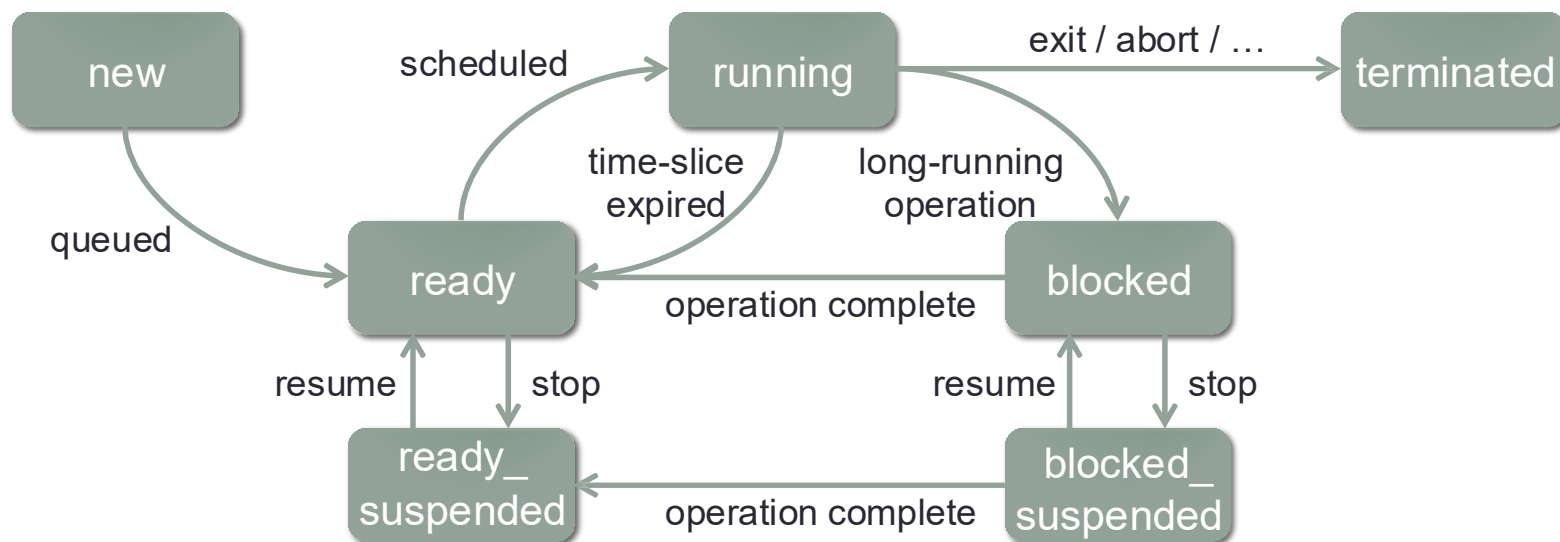

The process lifecycle

- Frequently, other states are also introduced to provide additional capabilities
- A common feature is the ability to suspend/resume processes
 - A suspended process will not be scheduled until it is resumed
 - A user can suspend a process with e.g., Ctrl-Z at command shell
 - A process can send a `SIGSTOP` signal to another process to suspend it



The process lifecycle

- The process being suspended might also have been blocked on a long-running operation
 - These extended states are conceptual and OS-dependent



Summary

- **Processes are a core abstraction in modern operating systems**
 - Each process has its own private address space, execution context, and associated system resources
- **The OS manages multiple processes concurrently**
 - Through CPU scheduling, context switching, and resource management
 - Creating the illusion that many processes run simultaneously
- **During their lifetime, processes move between several states**
 - New – process is being created
 - Ready – waiting to be scheduled on a CPU
 - Running – currently executing on a CPU
 - Blocked – waiting for an event (e.g., I/O completion)
 - Terminated – process has finished execution

Summary

■ Multiple strategies exist to handle concurrency

- Process creation (forking)
 - Uses blocking operations and a simple sequential programming model
 - Easier to implement and reason about
- Asynchronous (event-driven) I/O
 - Handles many connections within a single process
 - Improves scalability, but increases implementation complexity

■ Client/server architectures illustrate these trade-offs

- Process-based servers
 - Favor simplicity and strong isolation between clients
- Event-driven servers
 - Favor scalability and support for large numbers of concurrent connections